

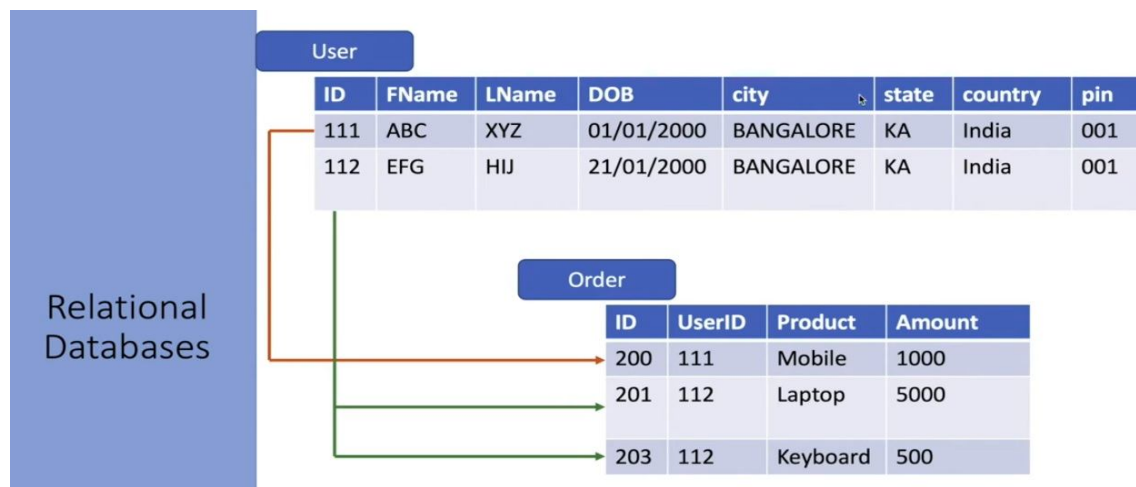
What is NoSQL. Explain its types.?

NoSql database means “Not Only SQL”, Non-Relational, Non tabular database and Schema less database.

Comparison with SQL and NoSQL.

Feature	SQL	NoSQL
Schema	Fixed	Flexible
Scalability	Vertical	Horizontal
Data type	Structured	Semi / Unstructured
Transactions	ACID	BASE (Basically Available, Soft State, Eventually Consistent)

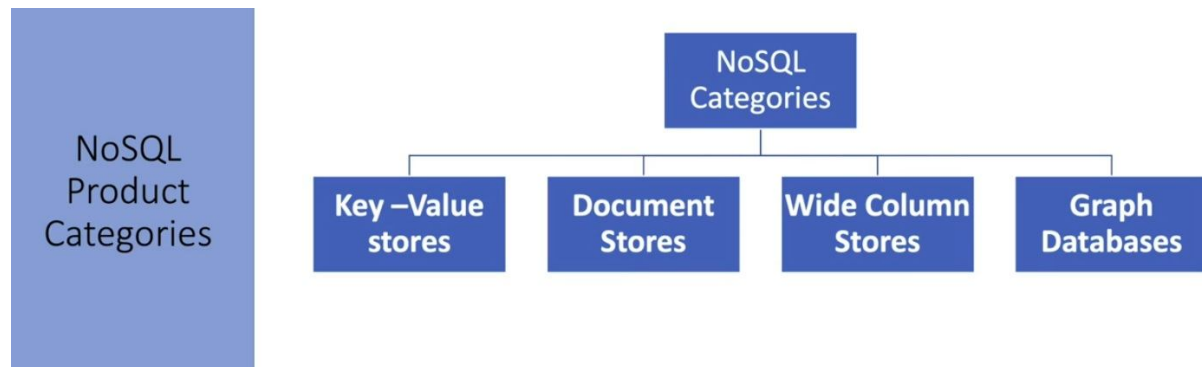
Relational Database



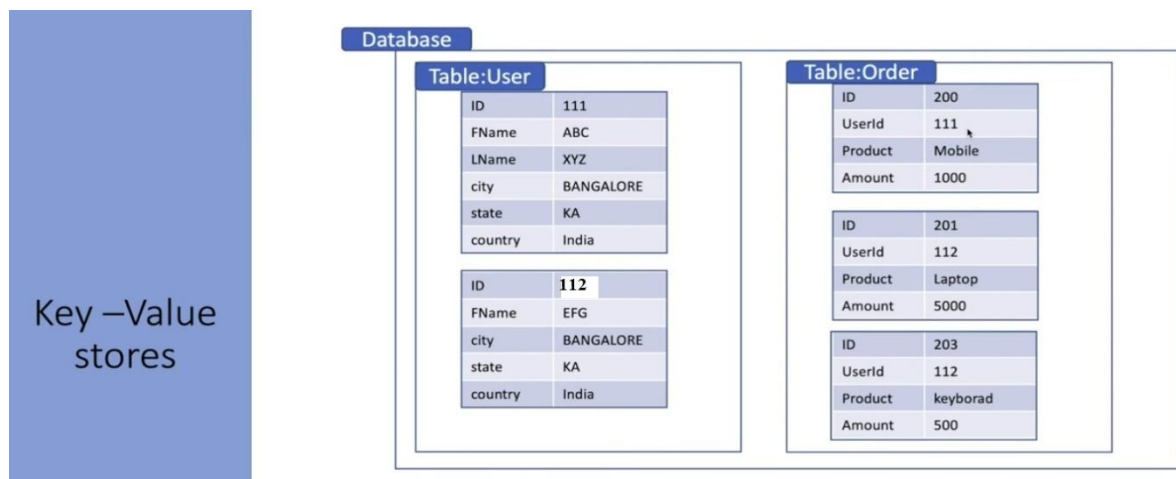
NOSQL



Types of NoSQL Databases



1. Key Value Stores: Data is stored as **key-value pairs**, Very fast and simple, Best for Caching, Session Data



Example structure:

Key → Value

ID → 111

Examples:

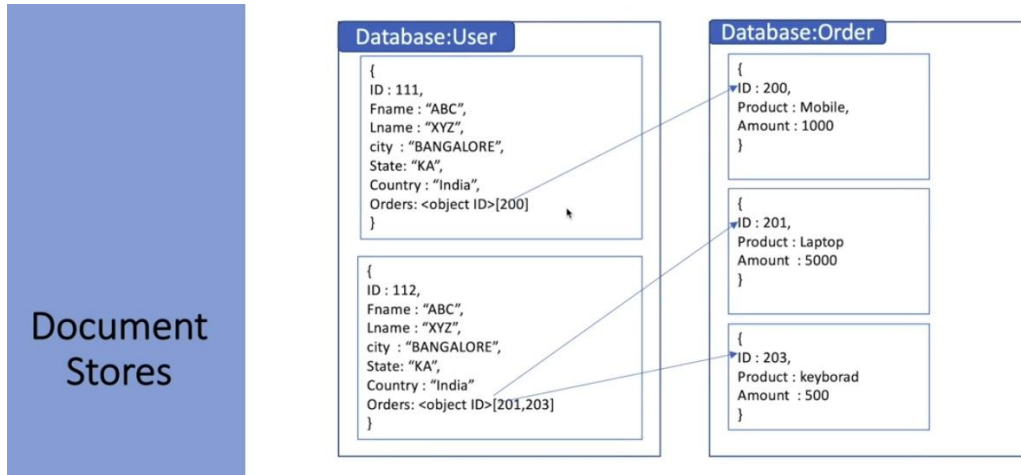
- Redis
- DynamoDB
- Riak

Use cases:

- Caching
- User sessions
- Shopping carts

2. Document-Oriented Database

- Data stored as **documents** (JSON, BSON, XML)
- Flexible schema
- Each document can have different fields



Example:

```
{
  "ID": 111,
  "FName": "ABC",
  "LName": "XYZ",
  "City": "BANGALORE",
  "State": "KA",
  "Country": "India",
  "Order": ["201", "203"]
}
```

Examples:

- MongoDB
- CouchDB
- Firestore

Use cases:

- Content management
- Web applications
- APIs

Comparison Between Key Value Store and Document Store

Feature	Key-Value Store	Document Store
Data Format	Key → Value (simple)	Key → JSON Document
Structure Awareness	No	Yes
Query Capability	By key only	By key + fields
Indexing	Usually key only	Multiple fields
Complexity	Very simple	More flexible
Example	Redis	MongoDB

3. Wide Column Store

- Data stored in **columns instead of rows**,
- Optimized for large datasets and analytics

Wide Column Stores

Table:User

ID	111	
Super Column : Name		
Column : FName	ABC	
Column : LName	XYZ	
Super Column : Address		
Column : city	BANGALORE	
Column : state	KA	
Column : country	India	
Super Column : Orders		
Orders	[200, 201]	

ID	111	
Super Column : Name		
Column : FName	ABC	
Column : LName	XYZ	

Table:Order

ID	200	
Super Column : PRODUCT		
Column : Product	Laptop	
Super Column : Price		
Column : Amount	5000	

ID	201	
Super Column : PRODUCT		
Column : Product	Keyboard	
Super Column : Price		
Column : Amount	500	

Examples:

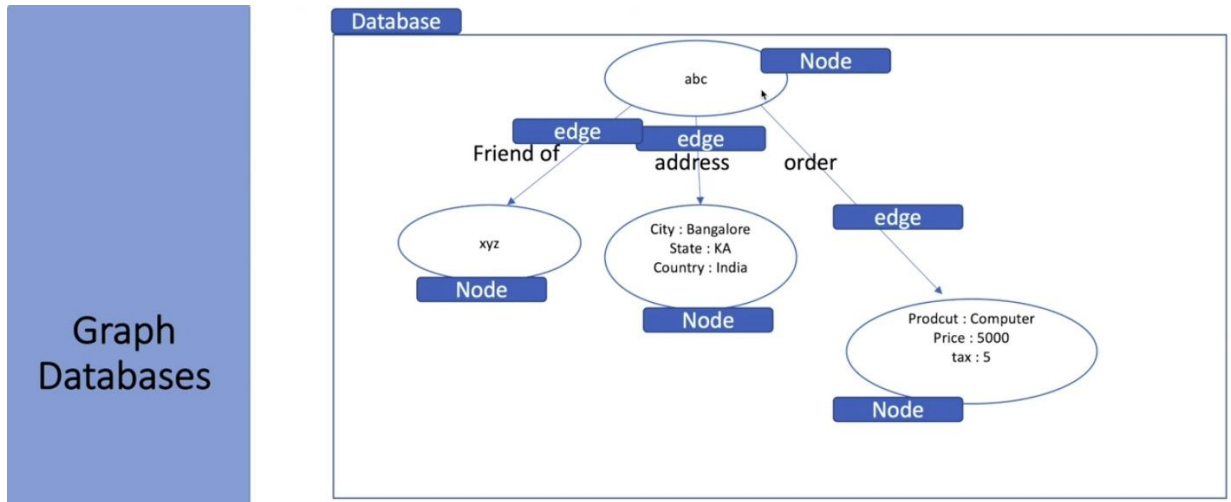
- Apache Cassandra
- HBase
- Google Bigtable

Use cases:

- Big data processing
- Time-series data
- Data warehousing

4. Graph Database:

- Data stored as **nodes and edges**
- Focuses on relationships



Examples:

- Neo4j
- Amazon Neptune
- ArangoDB

Use cases:

- Social networks
- Recommendation systems
- Fraud detection

Mongo DB

<https://www.youtube.com/watch?v=tw-wgbNP0cA&t=624s>

- `Mongosh`
- `Use database_name`
- `Show dbs`

1. Creation of database and Collection

- > `Use database_name`
- > `db.createCollection("myCollection")`
- > `show dbs`
- > `show collection`

1. Crud Operation

1. Create

- `.insertOne()`
- `.insertMany()`

2. Read

- `.find()`
- `.findOne()`

3. Update

- `.updateOne()`
- `.updateMany()`

4. Delete()

- `.deleteOne()`
- `.deleteMany()`

1. Create()

// insertOne()

```
company>db.user.insertOne(
{
  "fname": "Amit",
  "lname": "Kumar",
  "age": 25,
  "email": "amit@gmail.com",
  "mobile_no": 965841223,
  "address": {
    "house_no": 42,
    "nagar": "Saket",
    "city": "Indore",
    "state": "M.P.",
    "pin_code": 452001
  },
  "hobbies": ["Cricket", "Football"]
})
```

Case 1: Inserting Single Document

```
company> db.user.insertOne(
... {
...   "fname": "Amit",
...   "lname": "Kumar",
...   "age": 25,
...   "email": "amit@gmail.com",
...   "mobile_no": 965841223
... }
... )
{
  acknowledged: true,
  insertedId: ObjectId('6999725204ccf058f7628ca2')
}
```

Case 2: Inserting Nested Document

```
company> db.user.insertOne(
... {
...   "fname": "Amit",
...   "lname": "Kumar",
...   "age": 25,
...   "email": "amit@gmail.com",
...   "mobile_no": 965841223,
...   "address": {
...     "house_no": 42,
...     "nagar": "Saket",
...     "city": "Indore",
...     "state": "M.P.",
...     "pin_code": 452001
...   }
... }
... )
{
  acknowledged: true,
  insertedId: ObjectId('6999741f04ccf058f7628ca3')
}
```


Case 3: Inserting Array

```
company> db.user.insertOne(
... {
...   "fname": "Amit",
...   "lname": "Kumar",
...   "age": 25,
...   "email": "amit@gmail.com",
...   "mobile_no": 965841223,
...   "address": {
...     "house_no": 42,
...     "nagar": "Saket",
...     "city": "Indore",
...     "state": "M.P.",
...     "pin_code": 452001
...   },
...   "hobbies": ["Cricket", "Football"]
... }
... )
{
  acknowledged: true,
  insertedId: ObjectId('699974d804ccf058f7628ca4')
}
```

//insertMany[{}, {}, {}] Like that

```
company>db.user.insertOne(
[
  {
    "fname": "Sumit",
    "lname": "Gupta",
    "age": 20,
    "email": "sumit@gmail.com",
    "mobile_no": 9826433542,
    "address": {
      "house_no": 32,
      "nagar": "Vijay",
      "city": "Indore",
      "state": "M.P.",
      "pin_code": 452001
    },
    "hobbies": ["Chess", "Kabbadi"]
  },
  {
    "fname": "Deepak",
    "lname": "Mishra",
    "age": 35,
    "email": "deepak@gmail.com",
    "mobile_no": 9977562389,
    "address": {
      "house_no": 142,
      "nagar": "LIG",
      "city": "Indore",
      "state": "M.P.",
      "pin_code": 452001
    }
  }
]
```

```
    },  
    "hobbies": ["Cricket", "Football"]  
  },  
  {  
    "fname": "Vinit",  
    "lname": "Pal",  
    "age": 27,  
    "email": "vinit@gmail.com",  
    "mobile_no": 9425568923,  
    "address": {  
      "house_no": 36,  
      "nagar": "MIG",  
      "city": "Indore",  
      "state": "M.P.",  
      "pin_code": 452001  
    },  
    "hobbies": ["Cricket", "Boxing"]  
  }  
]  
)
```

2. Read()

1. find()

```
Company>db.user.find() //access all data
Company>db.user.find({hobbies:"Cricket"}) // Case Sensitive
company> db.user.find({"address.house_no":36})
company> db.users.findOne({ hobbies: "Cricket" }); //access single
```

3. Update()

1. updateOne()

//Update Existing Value

```
company> db.user.updateOne(
... {fname:"Amit"},
... {$set:{Age:25}}
... )
```

//Add a New Key and Value

```
company> db.user.updateOne(
... {fname:"Amit"},
... {$set:{Phone_No:07312761358}}
... )
```

//Add a New Value into Array

```
company> db.user.updateOne(
... {fname:"Amit"},
... {$push:{hobbies:"Running"}}
... )
```

//Extract a Value from Array

```
company> db.user.updateOne(  
... {fname:"Amit"},  
... {$pull:{hobbies:"Chess"}}  
... )
```

2. updateMany()

// Change done in Many Collections

```
company> db.user.updateMany(  
... {fname:"Amit"},  
... {$set:{Nationality:"Indian"}}  
... )
```

// Change done in Many Collections

```
company> db.user.updateMany(  
... {},  
... {$set:{Gender:"Male"}}  
... )
```

// Add a New Collections what match case not found

```
company> db.user.updateMany(  
    {fname:"Kamal"},  
    {$set:{Age:30, Mobile_No:9165841223, }},  
    {upsert:true}  
    )
```

4. Delete()

1. deleteOne()

Company>db.user.deleteOne({fname:"Kamal"}) // Case Sensitive

2. deleteMany()

Company>db.user.deleteMany({fname:"Kamal"}) // Case Sensitive

3. drop collection

Company>db.collection_name.drop()

Compony>db.order.drop()

4. drop database

Company>db.dropDatabase()